Machine Learning

1. Setting the scene: definitions & objectives

Louis Bagot
Université Claude Bernard Lyon 1

September 22, 2026

Course info

Websites & where to find info

- All administrative information on the course website
- Slides and labs for my part on the github, Machine Learning and Deep Learning parts
- All slides come with a “presentation mode” (the one I show in class) and a “reading mode” (with slides with notes) to read at home

Course sources

Course written by Louis Bagot.
Inspired by the following courses:

- Remy Cazabet, Sorbonne Université & Univ Lyon 1 (last year's course)
- Probabilistic Machine Learning, Kevin P. Murphy
- Understanding Deep Learning, by Simon J.D. Prince
- Machine Learning in Python with scikit-learn, by INRIA

See the slides with notes for references of individual slides (figures and notions).

What is Machine Learning?

What is Machine Learning?

Vague definition (Tom Mitchell):

> **A Computer Program is said to learn** from Experience E
> with respect to some class of Tasks T, and Performance measure P,
> **if its Performance at Tasks** in T, as measured by P,
> **improves with Experience** E.

What is Machine Learning?

Vague definition (Tom Mitchell):

> **A Computer Program is said to learn** from Experience E
> with respect to some class of Tasks T, and Performance measure P,
> **if its Performance at Tasks** in T, as measured by P,
> **improves with Experience** E.

$\rightarrow$ What are E, T, P, and the Computer Program ??

What are the Experience E, Task T, Performance P?

→ **Experience E =**

What are the Experience E, Task T, Performance P?

$\rightarrow$ **Experience E = DATA**

What are the Experience E, Task T, Performance P?

$\rightarrow$ Experience E = DATA

Data: the fundamental starting point of Machine Learning.

- Contains a bunch of *information* about something.

What are the Experience E, Task T, Performance P?

$\rightarrow$ Experience E = DATA

Data: the fundamental starting point of Machine Learning.

- Contains a bunch of *information* about something.
  - a bunch of text, images, patient or consumer information, scientific experiments, industrial processes, statistics on traffic or behaviors ...

What are the Experience E, Task T, Performance P?

$\longrightarrow$ Experience E = DATA

Data: the fundamental starting point of Machine Learning.

- Contains a bunch of *information* about something.
  - a bunch of text, images, patient or consumer information, scientific experiments, industrial processes, statistics on traffic or behaviors ...
  - $\rightarrow$ it's *because* the concept of data is vague/general that ML is powerful

What are the Experience E, Task T, Performance P?

$\rightarrow$ Experience E = DATA

Data: the fundamental starting point of Machine Learning.

- Contains a bunch of *information* about something.
  - a bunch of text, images, patient or consumer information, scientific experiments, industrial processes, statistics on traffic or behaviors ...
  - $\rightarrow$ it's *because* the concept of data is vague/general that ML is powerful
- Loosely, we want to extract *knowledge* from the data (defined with T, P)

Experience E = DATA, but what is it?

Experience E = DATA, but what is it?

Generally comes as a

$$\textbf{Dataset } \mathcal{D} = \{\boldsymbol{x_i}\}_{i=1}^{N}$$

- each $\boldsymbol{x_i}$ is a **sample** (e.g., a person, image, sound, company, ...)

Experience E = DATA, but what is it?

Generally comes as a

$$\textbf{Dataset } \mathcal{D} = \{\boldsymbol{x_i}\}_{i=1}^{N}$$

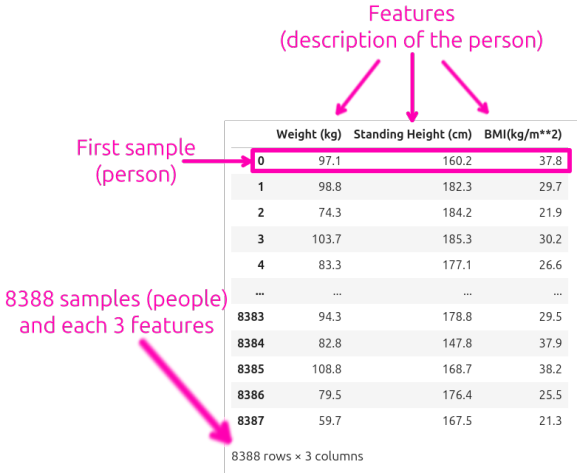
- each $\boldsymbol{x_i}$ is a **sample** (e.g., a person, image, sound, company, ...)
- each sample is a vector with several **features**: $\boldsymbol{x_i} = \left\{x_i^j\right\}_{j=1}^{d}$
  the features describe the sample (e.g. height, weight; colors, price, ...)

Experience E = DATA, what does it look like?

Experience E = DATA, what does it look like?

Dataset as a **matrix**:

(here, with the weight & height of people)



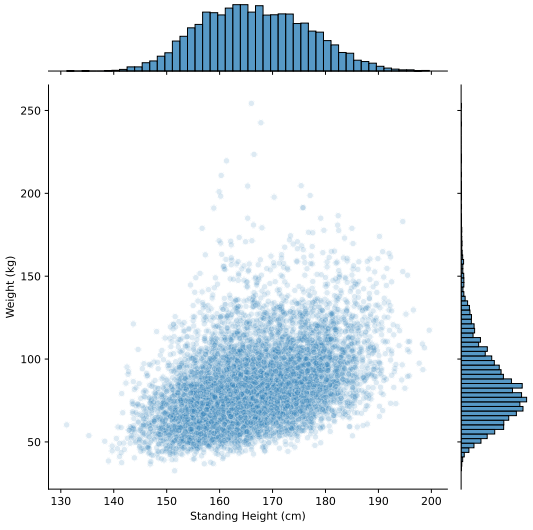
	Weight (kg)	Standing Height (cm)	BMI(kg/m**2)
0	97.1	160.2	37.8
1	98.8	182.3	29.7
2	74.3	184.2	21.9
3	103.7	185.3	30.2
4	83.3	177.1	26.6
...	...	...	...
8383	94.3	178.8	29.5
8384	82.8	147.8	37.9
8385	108.8	168.7	38.2
8386	79.5	176.4	25.5
8387	59.7	167.5	21.3

8388 rows × 3 columns

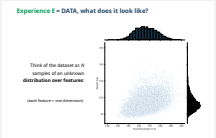
Experience E = DATA, what does it look like?

Think of the dataset as N samples of an unknown **distribution over features**:

(each feature = one dimension)



Notes for this slide



Think of each point (in a discrete space) or each small neighborhood region (in a continuous space like in the plot) as having a certain probability to appear under the "true" (but unknown) distribution. The dataset is just sampling *N* points from this "true" distribution.

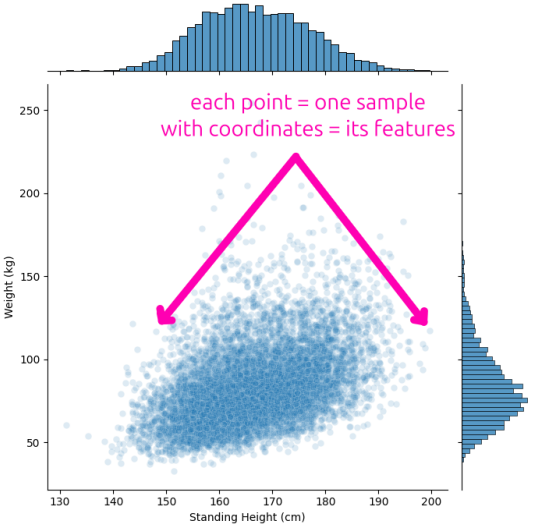
When doing probabilities in English, "sampling" refers to taking one element out of a probability function, for example, rolling a dice once is *sampling* from the uniform distribution that assigns $1/6$ proba to each face of the dice. When sampling from a distribution, what we obtain (for example, a 4) is a *sample*, hence the name of the rows in Machine Learning.

In this plot, we are actually doing the opposite: we use an existing dataset (height-weight dataset) and estimate the "true" distribution behind it. The estimation comes from the data itself, so we will see it's already a form of Machine Learning.

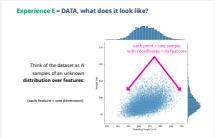
Experience E = DATA, what does it look like?

Think of the dataset as N samples of an unknown **distribution over features**:

(each feature = one dimension)



Notes for this slide



Think of each point (in a discrete space) or each small neighborhood region (in a continuous space like in the plot) as having a certain probability to appear under the "true" (but unknown) distribution. The dataset is just sampling *N* points from this "true" distribution.

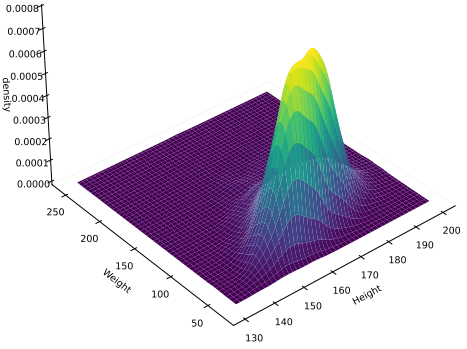
When doing probabilities in English, "sampling" refers to taking one element out of a probability function, for example, rolling a dice once is *sampling* from the uniform distribution that assigns $1/6$ proba to each face of the dice. When sampling from a distribution, what we obtain (for example, a 4) is a *sample*, hence the name of the rows in Machine Learning.

In this plot, we are actually doing the opposite: we use an existing dataset (height-weight dataset) and estimate the "true" distribution behind it. The estimation comes from the data itself, so we will see it's already a form of Machine Learning.

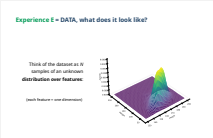
Experience E = DATA, what does it look like?

Think of the dataset as N samples of an unknown **distribution over features**:

(each feature = one dimension)



Notes for this slide



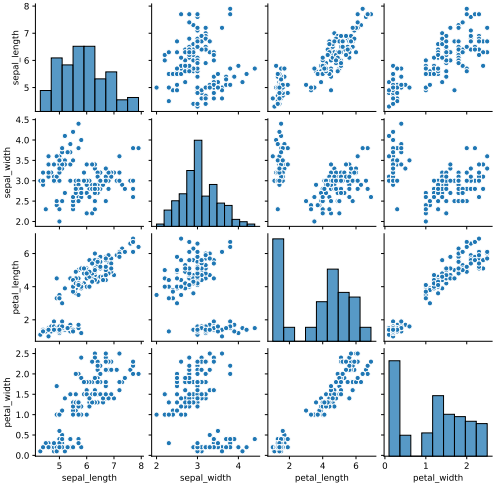
Think of each point (in a discrete space) or each small neighborhood region (in a continuous space like in the plot) as having a certain probability to appear under the "true" (but unknown) distribution. The dataset is just sampling *N* points from this "true" distribution.

When doing probabilities in English, "sampling" refers to taking one element out of a probability function, for example, rolling a dice once is *sampling* from the uniform distribution that assigns $1/6$ proba to each face of the dice. When sampling from a distribution, what we obtain (for example, a 4) is a *sample*, hence the name of the rows in Machine Learning.

In this plot, we are actually doing the opposite: we use an existing dataset (height-weight dataset) and estimate the "true" distribution behind it. The estimation comes from the data itself, so we will see it's already a form of Machine Learning.

Experience E = DATA, what does it look like?

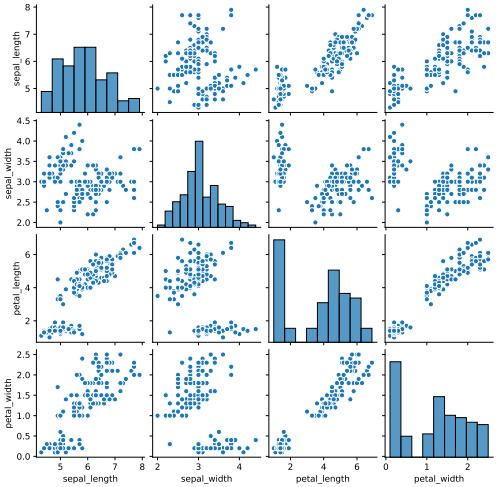
With more features (dimensions),
it becomes hard to visualize:



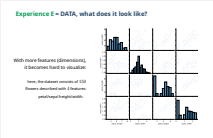
Experience E = DATA, what does it look like?

With more features (dimensions),
it becomes hard to visualize:

here, the dataset consists of 150
flowers described with 4 features:
petal/sepal height/width.



Notes for this slide



This dataset contains information about flowers: the petal and sepal (kinda sub-petal) length and width, for a total of 150 flowers and 4 features (4 dimensions).
We can't visualize data in 4D (even 3D is difficult on a screen) so instead we plot all the scatterplots (feature against feature), this is called a *pairplot*:

- (first row) x^1 with x^2, x^1 with x^3, x^1 with x^4,
- (second row) x^2 with x^3, x^2 with x^4
- (third row) x^3 with x^4
- it's useful to see the "marginal" distributions (like solo height and weight before) to see the overall shape of a feature, so we plot that in the x^1 with x^1 slot, same for all features (lands on the diagonal)
- by construction the plot is symmetrical: the bottom left triangle is the same as the upper right, transposed. (flipped along the diagonal axis $x = x$)

Never rely on 1-value metrics (mean, median, variance, modes ...) to summarize a distribution! Too much information lost.

What are the Experience E, Task T and Performance P?

What are the Experience E, Task T and Performance P?

→ What is the Task T?

What are we even trying to do with all that data?

What is the Task T?

Machine Learning:

Match a desired function, $f \approx g : \mathcal{X} \to \mathcal{Y}$

What is the Task T?

Machine Learning:

Match a desired function, $f \approx g : \mathcal{X} \to \mathcal{Y}$

In physics, if g is the true "laws of the world" to predict the future (gravity, electromagnetism, collisions ...) then f is called a **model** to approximate g.

$\to$ in ML we steal this name 😁

What is the Task T?

Machine Learning:

Match a desired function, $f \approx g : \mathcal{X} \to \mathcal{Y}$

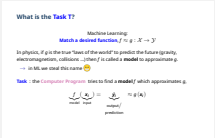
In physics, if g is the true "laws of the world" to predict the future (gravity, electromagnetism, collisions ...) then f is called a **model** to approximate g.

$\rightarrow$ in ML we steal this name 😁

Task : the **Computer Program** tries to find a **model** f which approximates g,

$$\underbrace{f}_{\text{model}}\left(\underbrace{\boldsymbol{x_i}}_{\text{input}}\right) = \underbrace{\hat{\boldsymbol{y}}_{\boldsymbol{i}}}_{\substack{\text{output} \\ \text{prediction}}} \approx g\left(\boldsymbol{x_i}\right)$$

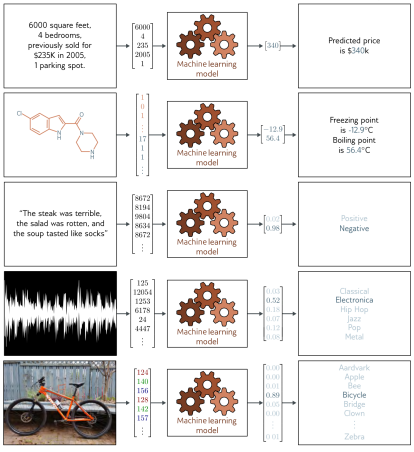
Notes for this slide

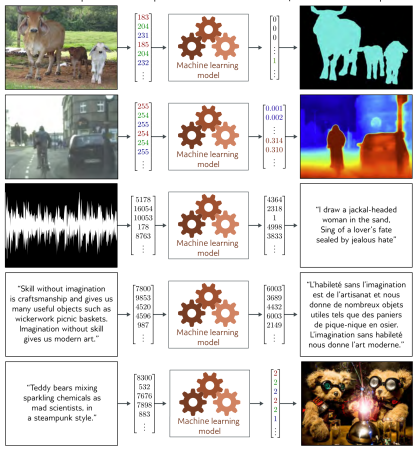


- $\boldsymbol{x_i}$ is then called the **input** to the model
- $f(\boldsymbol{x_i}) = \hat{\boldsymbol{y}}_{\boldsymbol{i}}$ is called the **output** or **prediction** (name comes from tasks where we "predict" some future value),
- the operation of computing the output (applying the sequence of mathematical operations that is f) is called **inference**.

Examples of functions g to approximate, please?

Examples of functions g to approximate, please?





Notes for this slide



Figure adapted from Understanding Deep Learning, Simon J.D. Prince. (see references at the start).

The original figure only mentioned Deep Learning model, but for now we can think of any transformation from model inputs to model outputs.

The original input information may come in a wide variety of forms, usually digitalized and stored numerically after being gathered from some real process (images, sound, log of observations...) ("real world input").

This needs to be turned into a model-readable input, containing the original information in the most concise and structured way possible to make the task as simple as possible for the model. After the computation of the input through the model (inference), the resulting output similarly needs to be re-processed into human-readable information.

Machine Learning:

Match a desired function, $f \approx g : \mathcal{X} \to \mathcal{Y}$

Machine Learning:

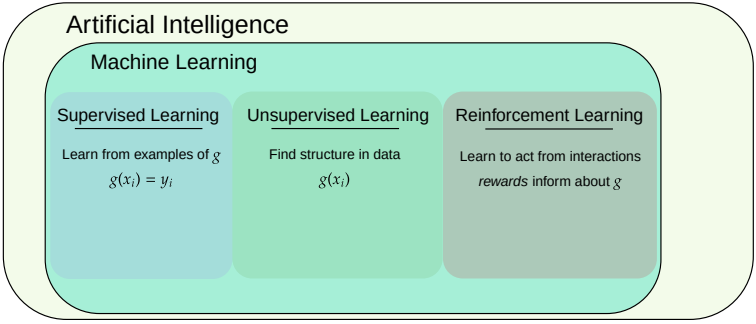
Match a desired function, $f \approx g : \mathcal{X} \to \mathcal{Y}$

...How do we get insights about g ?

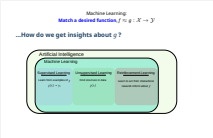
Machine Learning:

Match a desired function, $f \approx g : \mathcal{X} \to \mathcal{Y}$

...How do we get insights about g ?



Notes for this slide



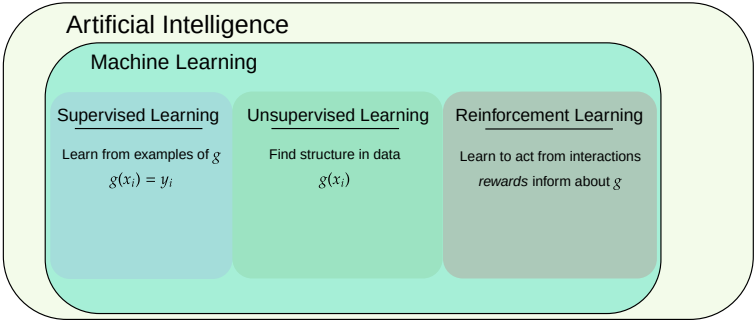
Adapted from *Understanding Deep Learning* by Simon J.D. Prince.

- *Supervised Learning*: learn to match inputs $\boldsymbol{x_i}$ and outputs $\boldsymbol{y_i}$.
  This typically requires a big dataset of labelled examples, e.g., pictures of animals and the name of the corresponding animal.
  $\rightarrow$ *g* is the "perfect" function with optimal outputs (matching the dataset)
- *Unsupervised Learning*: extracting information about a dataset of unlabelled data.
  This can take many forms: identifying clusters/groups, relationships between datapoints, key identifying features...
  $\rightarrow$ *g* contains this information (e.g. good clustering $g(\boldsymbol{x_i}) = \text{cluster } 3$)
- *Self-supervised* learning exists in-between supervised and unsupervised to describe problems where the model learns a task that did not require explicit labelling (image reconstruction, next token prediction)
- (RL on next slide note)

Machine Learning:

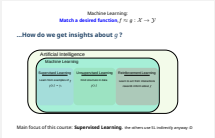
Match a desired function, $f \approx g : \mathcal{X} \to \mathcal{Y}$

...How do we get insights about g ?



Main focus of this course: **Supervised Learning.** the others use SL indirectly anyway :D

Notes for this slide



- *Reinforcement Learning*: problem of sequential decision making: the model (agent) needs to make a good sequence of decisions (games, robotics).
  As opposed to supervised learning, we do not know the answer (output label) to the question (input): for example in chess, i do not know the optimal move, i cannot teach a supervised model. Instead, the agent learns from interacting with the world, through trial and error.
  The objective is formulated as a reward function (e.g. $+1$ if I win, -1 if I lose). $\rightarrow g$ is the optimal policy, the behavior that gathers the most possible reward in all situations

Supervised Learning: “examples of g” ? Where?

Supervised Learning: "examples of g" ? Where?

In Supervised Learning, we extend to a **labelled dataset**

$$\mathcal{D} = \{\boldsymbol{x_i}, \boldsymbol{y_i}\}_{i=1}^{N}$$

Enhanced with **labels** $\boldsymbol{y_i} = g(\boldsymbol{x_i})$ giving the "answer". From where?

Supervised Learning: "examples of g" ? Where?

In Supervised Learning, we extend to a **labelled dataset**

$$\mathcal{D} = \{\boldsymbol{x_i}, \boldsymbol{y_i}\}_{i=1}^{N}$$

Enhanced with **labels** $\boldsymbol{y_i} = g(\boldsymbol{x_i})$ giving the "answer". From where?

- Generally provided by humans (captcha-like problems)
- Can be gathered from natural processes (historic observations e.g. climate?)

Supervised Learning: "examples of g" ? Where?

In Supervised Learning, we extend to a **labelled dataset**

$$\mathcal{D} = \{\boldsymbol{x_i}, \boldsymbol{y_i}\}_{i=1}^{N}$$

Enhanced with **labels** $\boldsymbol{y_i} = g(\boldsymbol{x_i})$ giving the "answer". From where?

- Generally provided by humans (captcha-like problems)
- Can be gathered from natural processes (historic observations e.g. climate?)
- Major headache of Supervised Learning! It's costly!

Supervised Learning: what do the labels look like?

Supervised Learning: what do the labels look like?

Two main types of Supervised Learning:

- **Regression**: y_i are real values, e.g. temperature, prices...
  - predict the price of an apartment from size, location, number of rooms...
  - predict the number of passengers on all metro lines
  - predict the distance to walls and to the goal in robotics

Supervised Learning: what do the labels look like?

Two main types of Supervised Learning:

- **Regression**: y_i are real values, e.g. temperature, prices...
  - predict the price of an apartment from size, location, number of rooms...
  - predict the number of passengers on all metro lines
  - predict the distance to walls and to the goal in robotics

- **Classification**: y_i is a **class**, an element within a list
  - yes or no (patient:illness? object:present? positive feeling?)
  - item on an image
  - grandmaster's move in a chess game

Supervised Learning: what do the labels look like?

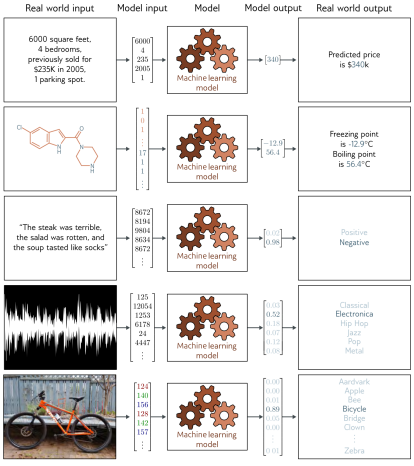
Two main types of Supervised Learning:

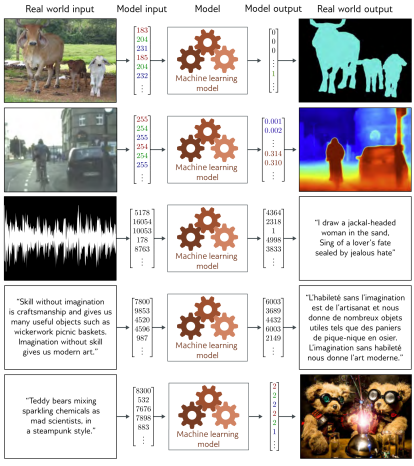
- **Regression**: y_i are real values, e.g. temperature, prices...
  - predict the price of an apartment from size, location, number of rooms...
  - predict the number of passengers on all metro lines
  - predict the distance to walls and to the goal in robotics

- **Classification**: y_i is a **class**, an element within a list
  - yes or no (patient:illness? object:present? positive feeling?)
  - item on an image
  - grandmaster's move in a chess game

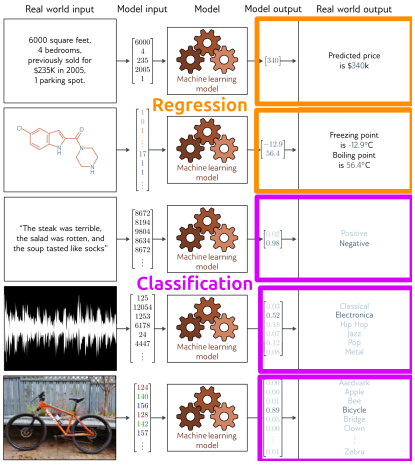
In all cases our Task is to approximate g

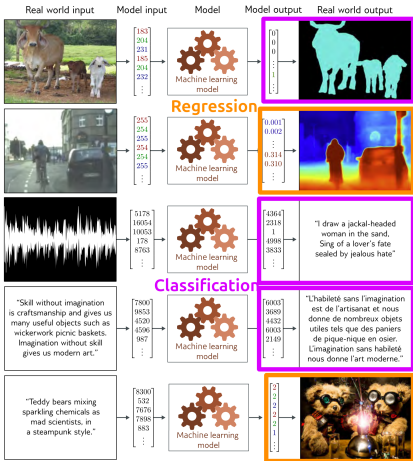
Classification vs Regression: can you guess?





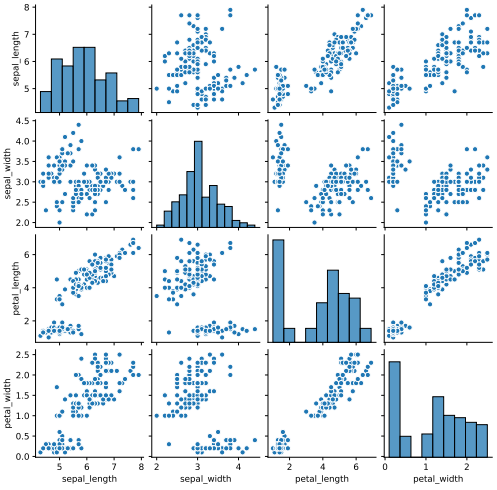
Classification vs Regression: can you guess?





What does it look like now?

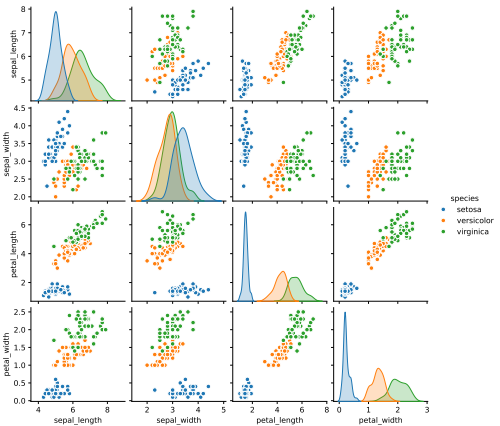
Still think of it as a distribution, but some of the columns / dimensions are hidden from the model:



What does it look like now?

Still think of it as a distribution, but some of the columns / dimensions are hidden from the model:

the flowers belong to 3 different species!



Supervised Learning: approximate g with the model f from examples:

$$f(\boldsymbol{x_i}) \approx \boldsymbol{y_i} = g(\boldsymbol{x_i}).$$

Supervised Learning: approximate g with the model f from examples:

$$f(\boldsymbol{x_i}) \approx \boldsymbol{y_i} = g(\boldsymbol{x_i}).$$

...What does it mean for a model to approximate g "well"?

$\rightarrow$ What is the Performance metric P?

Supervised Learning: approximate g with the model f from examples:

$$f(\boldsymbol{x_i}) \approx \boldsymbol{y_i} = g(\boldsymbol{x_i}).$$

...What does it mean for a model to approximate g "well"?

$\rightarrow$ What is the Performance metric P?

General idea:

Minimize distance between $f(\boldsymbol{x_i})$ and $\boldsymbol{y_i}$

What is the Performance metric P?

$\rightarrow$ **what is the notion of distance between $f(x_i)$ and y_i?**

What is the Performance metric P?

$\longrightarrow$ what is the notion of distance between $f(\boldsymbol{x_i})$ and y_i?

Simplest (Mean Squared Error) (we will see others later): if $y_i \in \mathbb{R}$:

$$\frac{1}{N}\sum_{i=1}^{N}\left(y_i - f(\boldsymbol{x_i})\right)^2$$

What is the Performance metric P?

$\rightarrow$ what is the notion of distance between $f(x_i)$ and y_i?

Simplest (Mean Squared Error) (we will see others later): if $y_i \in \mathbb{R}$:

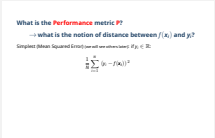
$$\frac{1}{N}\sum_{i=1}^{N}\left(y_i - f(\boldsymbol{x_i})\right)^2$$

What is the Performance metric P?

$\rightarrow$ what is the notion of distance between $f(\boldsymbol{x_i})$ and y_i?

Simplest (Mean Squared Error) (we will see others later): if $y_i \in \mathbb{R}$:

$$\frac{1}{N}\sum_{i=1}^{N}\left(y_i - f(\boldsymbol{x_i})\right)^2$$



Notes for this slide

For $\boldsymbol{y_i} \in \mathcal{Y} \subseteq \mathbb{R}^m$, use

$$\|\boldsymbol{y_i} - f(\boldsymbol{x_i})\|_1 \doteq \sum_{j=1}^{m} \left| (\boldsymbol{y_i})_j - f(\boldsymbol{x_i})_j \right|$$

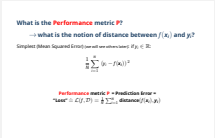
What is the Performance metric P?

$\rightarrow$ what is the notion of distance between $f(\boldsymbol{x_i})$ and $\boldsymbol{y_i}$?

Simplest (Mean Squared Error) (we will see others later): if $y_i \in \mathbb{R}$:

$$\frac{1}{N}\sum_{i=1}^{N}\left(y_i - f(\boldsymbol{x_i})\right)^2$$

Performance metric P = Prediction Error =
"Loss" $\doteq \mathcal{L}(f, \mathcal{D}) = \frac{1}{N}\sum_{i=1}^{N}$ **distance**$(f(\boldsymbol{x_i}), \boldsymbol{y_i})$



Notes for this slide

We will study more types of loss / distances later on as we need them

What is the Performance metric P?

$\rightarrow$ what is the notion of distance between $f(\boldsymbol{x_i})$ and y_i?

Simplest (Mean Squared Error) (we will see others later): if $y_i \in \mathbb{R}$:

$$\frac{1}{N}\sum_{i=1}^{N}\left(y_i - f(\boldsymbol{x_i})\right)^2$$

Performance metric P = Prediction Error =
"Loss" $\doteq \mathcal{L}(f, \mathcal{D}) = \frac{1}{N}\sum_{i=1}^{N}$ **distance**$(f(\boldsymbol{x_i}), \boldsymbol{y_i})$

To maximize the Performance, we minimize the Loss

Summary time!

What is Machine Learning ?

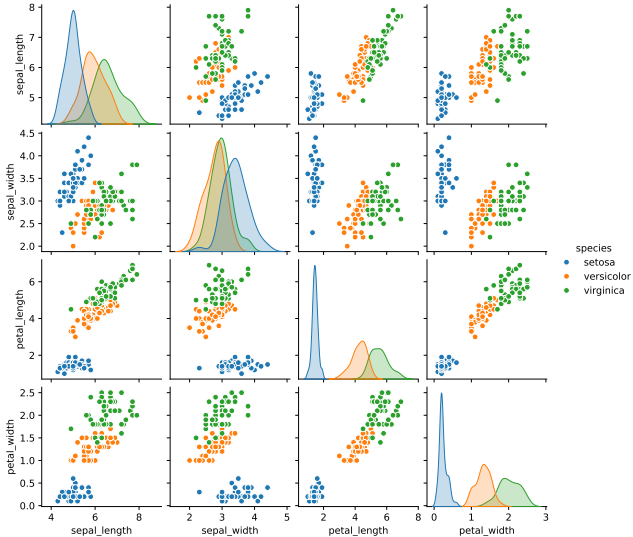
Summary time!

What is Machine Learning ?

The Computer Program learns a model to predict the labels from the Data, as measured by the decrease in prediction error it makes

The Computer Program learns a model... How? What is it?

How would **you** tackle it?



Just pick the label of the closest thing?

Just pick the label of the closest thing?

Yes!

Algo called **K-Nearest Neighbors** (KNN):

- pick a number of neighbors $k \geqslant 1$
- pick a notion of distance $\delta(\boldsymbol{x_i}, \boldsymbol{x_j})$

Just pick the label of the closest thing?

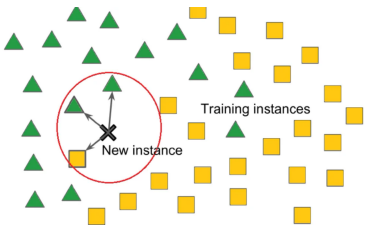
Yes!

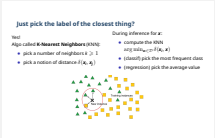
Algo called **K-Nearest Neighbors** (KNN):

- pick a number of neighbors $k \geqslant 1$
- pick a notion of distance $\delta(\boldsymbol{x_i}, \boldsymbol{x_j})$

During inference for $\boldsymbol{x}$:

- compute the KNN $\arg\min_{\boldsymbol{x_i} \in \mathcal{D}} \delta(\boldsymbol{x_i}, \boldsymbol{x})$
- (classif) pick the most frequent class
- (regression) pick the average value





Notes for this slide

Picture taken from the ML Archive KNN Explainer.

How good is KNN?

How good is KNN?

K-Nearest Neighbors analysis:

- simple, can be surprisingly good
- explainable $\rightarrow$ clear strengths and weaknesses

How good is KNN?

K-Nearest Neighbors analysis:

- simple, can be surprisingly good
- explainable $\rightarrow$ clear strengths and weaknesses
- ultra-reliant on a good distance
- needs to pass through the entire dataset at inference!
- ... did we learn anything? good baseline though!

How do we avoid passing through the whole dataset?

How do we avoid passing through the whole dataset?

Parametrized models:

How do we avoid passing through the whole dataset?

Parametrized models: math. functions with two sets of inputs, $f_{\boldsymbol{\theta}}(\boldsymbol{x})$

- Usual inputs $\boldsymbol{x}$, e.g. a dataset sample, $\boldsymbol{x} = (x_1, x_2, \ldots, x_d)$, each $x_i \in \mathbb{R}$
- The **parameters** $\boldsymbol{\theta}$, a set of scalars, $\boldsymbol{\theta} = (\theta_1, \theta_2, \ldots, \theta_M)$, each $\theta_i \in \mathbb{R}$
  - (sometimes called **weights**)



Notes for this slide

Parametrized models are sometimes written $f(\boldsymbol{x}, \boldsymbol{\theta})$ as well.

How do we avoid passing through the whole dataset?

Parametrized models: math. functions with two sets of inputs, $f_{\boldsymbol{\theta}}(\boldsymbol{x})$

- Usual inputs $\boldsymbol{x}$, e.g. a dataset sample, $\boldsymbol{x} = (x_1, x_2, \ldots, x_d)$, each $x_i \in \mathbb{R}$
- The **parameters** $\boldsymbol{\theta}$, a set of scalars, $\boldsymbol{\theta} = (\theta_1, \theta_2, \ldots, \theta_M)$, each $\theta_i \in \mathbb{R}$
  - (sometimes called **weights**)

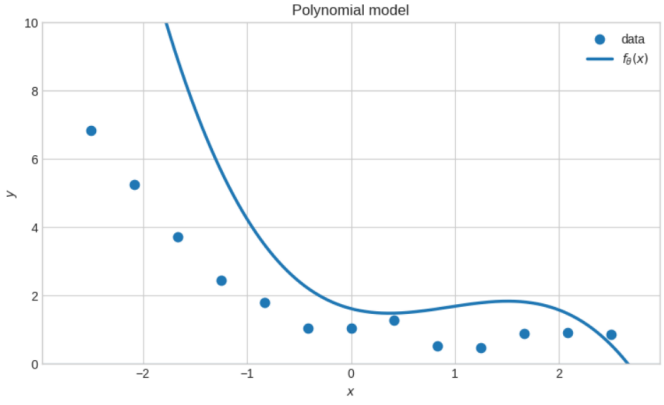
- **The parametrized model is just a big math. formula involving $\boldsymbol{x}$ and $\boldsymbol{\theta}$.**

How do we avoid passing through the whole dataset?

Parametrized models, simplest example: **a linear or polynomial model**

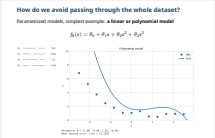
$$f_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

θ_0 1.60
θ_1 −0.80
θ_2 1.35
θ_3 −0.48



Parameters θ = [1.60, -0.80, 1.35, -0.48]
Mean squared error L(θ) = 21.2285

Notes for this slide

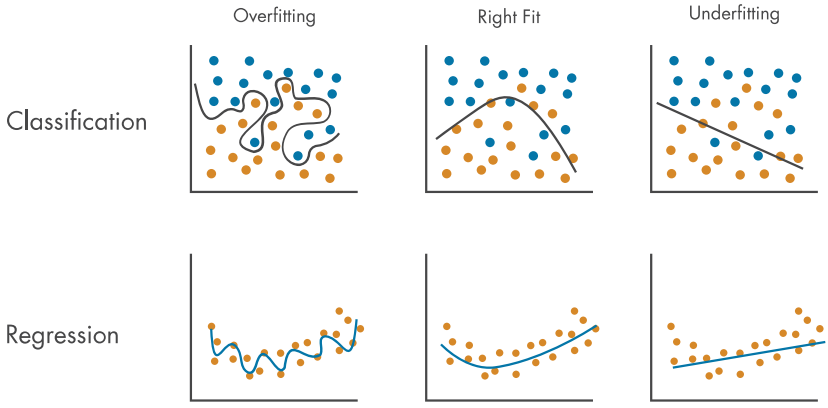


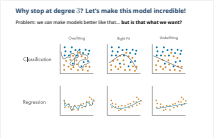
We call the model ”linear” if it only consists of multiplications with weights and additions with weighted terms together. We call it ”polynomial” if we also allow the model to contain terms of order x^2 or more, as well as terms multiplying inputs together for multi-input data.

Why stop at degree 3? Let's make this model incredible!

Why stop at degree 3? Let's make this model incredible!

Problem: we *can* make models better like that... **but is that what we want?**





Notes for this slide

plot from this blog post

“Overfitting”: But isn’t my loss minimal then?

“Overfitting”: But isn’t my loss minimal then?

Yes. Definition mistake: just minimizing the loss isn’t what we want

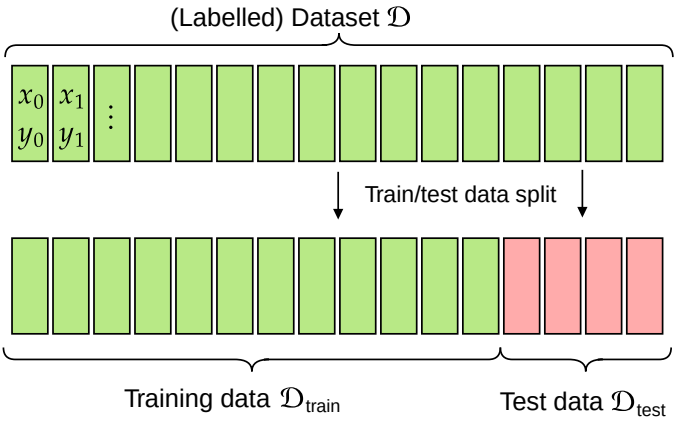
- the model can simply **memorize** the dataset examples!
- happens with too little data, too many parameters.

It’s in the **Task** and **Performance** metrics:

→ minimize the loss **on never-seen samples**!

How do we get samples we've never seen?

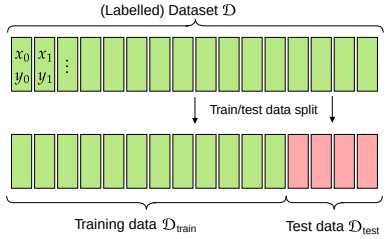
Implementation trick: split the dataset into test/train! (e.g. 30-70%)



How do we get samples we've never seen?

Implementation trick:
split the dataset into test/train!

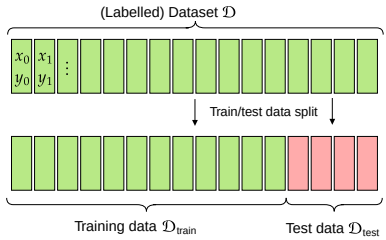
- **Train models on** $\mathcal{D}_{\text{train}}$
- **Evaluate & compare models on** $\mathcal{D}_{\text{test}}$



How do we get samples we've never seen?

Implementation trick:
split the dataset into test/train!

- **Train models on** $\mathcal{D}_{\text{train}}$
- **Evaluate & compare models on** $\mathcal{D}_{\text{test}}$



You know you're overfitting when the **training performance is much better than the testing one!**

Important note: scientific integrity

Important note: scientific integrity

Machine Learning will feel like cooking a lot of the time.

- picking models
- picking hyperparameters
- applying random tricks & hacks

Important note: scientific integrity

Machine Learning will feel like cooking a lot of the time.

- picking models
- picking hyperparameters
- applying random tricks & hacks

Model evaluation is where the real science happens.

Important note: scientific integrity

Machine Learning will feel like cooking a lot of the time.

- picking models
- picking hyperparameters
- applying random tricks & hacks

Model evaluation is where the real science happens.

- train/test is like the placebo or A/B test for ML
- it informs about the real outcome you can expect from the model.

Important note: scientific integrity

Machine Learning will feel like cooking a lot of the time.

- picking models
- picking hyperparameters
- applying random tricks & hacks

Model evaluation is where the real science happens.

- train/test is like the placebo or A/B test for ML
- it informs about the real outcome you can expect from the model.
- easy to mess up: **never re-use the test data to fine-tune models**
  - otherwise it's like picking the perfect treatment for 10 specific people
  - re-sample new test data before re-training
  - use a *validation set* for hyperparameter tuning

Where were we?

Where were we?

Improve from KNN $\longrightarrow$ **Parametrized models** $f_{\theta}(\boldsymbol{x})$

Where were we?

Improve from KNN $\longrightarrow$ **Parametrized models** $f_{\theta}(\boldsymbol{x})$

What are we trying to do with those?

Where were we?

Improve from KNN $\rightarrow$ **Parametrized models** $f_{\boldsymbol{\theta}}(\boldsymbol{x})$

What are we trying to do with those?

Think of all the possible curves you can generate with your parametrized model, by changing the parameters $\boldsymbol{\theta} = (\theta_1, \theta_2, \dots)$.

Where were we?
Improve from KNN $\rightarrow$ **Parametrized models** $f_{\boldsymbol{\theta}}(\boldsymbol{x})$

What are we trying to do with those?

Think of all the possible curves you can generate with your parametrized model, by changing the parameters $\boldsymbol{\theta} = (\theta_1, \theta_2, \dots)$.

We are looking for **the curves that best fits the data**
$\rightarrow$ the set of parameters $\boldsymbol{\theta}$ with the lowest loss:

$$\arg\min_{\boldsymbol{\theta}} \mathcal{L}(f_{\boldsymbol{\theta}}, \mathcal{D})$$

($\mathcal{D}_{\text{train}}$ for training, $\mathcal{D}_{\text{test}}$ for eval)

What does that look like?

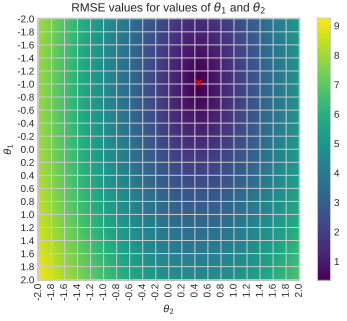
New change of perspective:

Visualize the **weights as dimensions**, (all values each weight can take in $\mathbb{R}$),

the **loss as a surface / color gradient**.

$\rightarrow$ We are looking for the lowest spot!

$$\arg\min_{\boldsymbol{\theta}} \mathcal{L}(f_{\boldsymbol{\theta}}, \mathcal{D}_{\text{test}})$$



[click] cool interactive version

$$\arg\min_{\boldsymbol{\theta}} \mathcal{L}(f_{\boldsymbol{\theta}}, \mathcal{D}_{\text{test}})$$

This is called an **optimization problem**

$\rightarrow$ we're looking for the *optimal* parameters, $\boldsymbol{\theta}^\star$

How do we find the optimal parameters?

$$\arg\min_{\boldsymbol{\theta}} \mathcal{L}(f_{\boldsymbol{\theta}}, \mathcal{D}_{\text{test}})$$

This is called an **optimization problem**

$\rightarrow$ we're looking for the *optimal* parameters, $\boldsymbol{\theta}^\star$

How do we find the optimal parameters?

Many approaches possible: (we will see some, also see optim course)

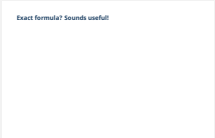
- exact formula
- random shooting
- genetic algorithms
- gradient-based methods
- ...

Notes for this slide



Different methods have different applicability ranges, advantages and disadvantages depending on the context.

Exact formula? Sounds useful!

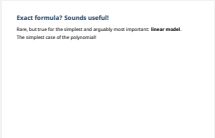


Notes for this slide

We don't use the polynomial as much with multi-dimensional inputs because of the many, many cross-multiplication terms $x_i x_j$ which appear as soon as the order is $\geqslant 2$.
We *can* use them though, and if you have a reason to believe it would be useful, definitely do it!
This is called an *inductive bias*: you incorporate your understanding of the problem in the model structure itself.

Exact formula? Sounds useful!

Rare, but true for the simplest and arguably most important: **linear model**.
The simplest case of the polynomial!



Notes for this slide

In the second formula in vector form, we extend $\boldsymbol{x}$ to contain a constant first term $x_0 = 1$. This is convenient for the notation, but also meaningful: if all the inputs are at 0, it could still be possible to output a non-zero number (the weight of a basket of fruits & vegetables if the basket is empty)

Exact formula? Sounds useful!

Rare, but true for the simplest and arguably most important: **linear model**.
The simplest case of the polynomial!

For an input with many dimensions, weight each input with some scalar weight:

$$\begin{aligned} f_{\boldsymbol{\theta}}(\boldsymbol{x}) &= \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots \\ &= \boldsymbol{x}^\top \boldsymbol{\theta} \end{aligned}$$

Exact formula? Sounds useful!

Rare, but true for the simplest and arguably most important: **linear model**.
The simplest case of the polynomial!

For an input with many dimensions, weight each input with some scalar weight:

$$\begin{aligned} f_{\boldsymbol{\theta}}(\boldsymbol{x}) &= \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots \\ &= \boldsymbol{x}^\top \boldsymbol{\theta} \end{aligned}$$

Writing the dataset design matrix X and dataset label vector $\boldsymbol{y}$, we can compute exactly the optimal (problem and solution called **Ordinary Least Squares**)

$$\boldsymbol{\theta}^\star \doteq \arg\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_i (\boldsymbol{x_i}^\top \boldsymbol{\theta} - y_i)^2 = (X^\top X)^{-1} X \boldsymbol{y}$$

Drawing lines? Isn’t that too simplistic?

Drawing lines? Isn’t that too simplistic?

Yes. But the simplicity is the point:

- super fast to train & run

Drawing lines? Isn't that too simplistic?

Yes. But the simplicity is the point:

- super fast to train & run
- many simple relationships are linear:
  - the price of n items where each item costs c is cn
  - physical quantities can similarly add up: mass, energy
  - in general, this encodes "a bit more x consistently gives a bit more/less y"

Drawing lines? Isn't that too simplistic?

Yes. But the simplicity is the point:

- super fast to train & run
- many simple relationships are linear:
  - the price of n items where each item costs c is cn
  - physical quantities can similarly add up: mass, energy
  - in general, this encodes "a bit more x consistently gives a bit more/less y"
- it informs about linear relationships:
  - if θ_j is 0, then feature x_j is linearly irrelevant
  - if θ_j is positive, y generally increases with x_j
  - if θ_j is negative, y generally decreases with x_j
  - $\rightarrow$ gives **insight**!! now we're learning!

Drawing lines? Isn't that too simplistic?

Yes. But the simplicity is the point:

- **understanding why and how we can expect the model to fail** is huge.
  - we understand its limits and what we should expect from it
  - the hypothesis is very clear:
    if the relationships are linear, this model will find how.

Drawing lines? Isn't that too simplistic?

Yes. But the simplicity is the point:

- **understanding why and how we can expect the model to fail** is huge.
  - we understand its limits and what we should expect from it
  - the hypothesis is very clear:
    if the relationships are linear, this model will find how.

- can make great predictions about out-of-distribution data
  - thanks to the linear assumptions, new data far from the dataset range can be estimated meaningfully

Drawing lines? Isn't that too simplistic?

Yes. But the simplicity is the point:

- **understanding why and how we can expect the model to fail** is huge.
  - we understand its limits and what we should expect from it
  - the hypothesis is very clear:
    if the relationships are linear, this model will find how.
- can make great predictions about out-of-distribution data
  - thanks to the linear assumptions, new data far from the dataset range can be estimated meaningfully
- this explains why **the linear model is still extremely popular**

Other losses/evaluation metrics

We have seen **Mean Squared Error**:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(f_{\boldsymbol{\theta}}(\boldsymbol{x_i}) - y_i)^2$$

- most popular loss
- sensitive to outliers (very heavy, squared weight)
- interpretation issue: big value, hard to read...
  $\rightarrow$ can use Root MSE $RMSE = \sqrt{\text{MSE}}$

Other losses/evaluation metrics

Use the R^2 **score** for readability:

$$R^2 = 1 - \frac{\text{MSE}}{\text{Variance}} = 1 - \frac{\sum_{i=1}^{N} (f_{\boldsymbol{\theta}}(\boldsymbol{x_i}) - y_i)^2}{\sum_{i=1}^{N} (\bar{\boldsymbol{y}} - y_i)^2}$$

- compares our prediction to the naive model "prediction = mean"
- perfect model: MSE=0, $R^2 = 1$
- naive model: MSE=Variance, $R^2 = 0$
- terrible model: $R^2 < 0$

Other losses/evaluation metrics

Mean Absolute Error:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^{N} |f_{\boldsymbol{\theta}}(\boldsymbol{x_i}) - y_i|$$

- easier to read than MSE (in the unit of y)
- less sensitive to outliers
- less punitive of errors